

Widget 构造函数可以做它想做的任何事。它有可能又 new 一些内存, 而没人可以强迫它再次使用 `nothrow new`。因此虽然 `"new (std::nothrow) Widget"` 调用的 `operator new` 并不抛掷异常, 但 Widget 构造函数却可能会。如果它真那么做, 该异常会一如往常地传播。需要结论吗? 结论就是: 使用 `nothrow new` 只能保证 `operator new` 不抛掷异常, 不保证像 `"new (std::nothrow) Widget"` 这样的表达式绝不导致异常。因此你其实没有运用 `nothrow new` 的需要。

无论使用正常 (会抛出异常) 的 `new`, 或是其多少有点发育不良的 `nothrow` 兄弟, 重要的是你需要了解 `new-handler` 的行为, 因为两种形式都使用它。

请记住

- `set_new_handler` 允许客户指定一个函数, 在内存分配无法获得满足时被调用。
- `Nothrow new` 是一个颇为局限的工具, 因为它只适用于内存分配; 后继的构造函数调用还是可能抛出异常。

条款 50: 了解 new 和 delete 的合理替换时机

Understand when it makes sense to replace new and delete.

让我们暂时回到根本原理。首先, 怎么会有人想要替换编译器提供的 `operator new` 或 `operator delete` 呢? 下面是三个最常见的理由:

- 用来检测运用上的错误。如果将“new 所得内存”delete 掉却不幸失败, 会导致内存泄漏 (memory leaks)。如果在“new 所得内存”身上多次 delete 则会导致不确定行为。如果 `operator new` 持有一串动态分配所得地址, 而 `operator delete` 将地址从中移走, 倒是很容易检测出上述错误用法。此外各式各样的编程错误可能导致数据 “overruns” (写入点在分配区块尾端之后) 或 “underruns” (写入点在分配区块起点之前)。如果我们自行定义一个 `operator new`, 便可超额分配内存, 以额外空间 (位于客户所得区块之前或后) 放置特定的 byte patterns (即签名, signatures)。operator deletes 便得以检查上述签名是否原封不动, 若否就表示在分配区的某个生命时间点发生了 overrun 或 underrun, 这时候 `operator delete` 可以志记 (log) 那个事实以及那个惹是生非的指针。

- **为了强化效能。**编译器所带的 `operator new` 和 `operator delete` 主要用于一般目的，它们不但可被长时间执行的程序（例如网页服务器，web servers）接受，也可被执行时间少于一秒的程序接受。它们必须处理一系列需求，包括大块内存、小块内存、大小混合型内存。它们必须接纳各种分配形态，范围从程序存活期间的少量区块动态分配，到大数量短命对象的持续分配和归还。它们必须考虑破碎问题（fragmentation），这最终会导致程序无法满足大块内存要求，即使彼时有总量足够但分散为许多小区块的自由内存。

现实存在这么些个对内存管理器的要求，因此编译器所带的 `operator new` 和 `operator delete` 采取中庸之道也就不令人惊讶了。它们的工作对每个人都是适度地好，但不对特定任何人有最佳表现。如果你对你的程序的动态内存运用型态有深刻的了解，通常可以发现，定制版之 `operator new` 和 `operator delete` 性能胜过缺省版本。说到胜过，我的意思是它们比较快，有时甚至快很多，而且它们需要的内存比较少，最高可省 50%。对某些（虽然不是所有）应用程序而言，将旧有的（编译器自带的）`new` 和 `delete` 替换为定制版本，是获得重大效能提升的办法之一。

- **为了收集使用上的统计数据。**在一头栽进定制型 `new` 和定制型 `delete` 之前，理当先收集你的软件如何使用其动态内存。分配区块的大小分布如何？寿命分布如何？它们倾向于以 FIFO（先进先出）次序或 LIFO（后进先出）次序或随机次序来分配和归还？它们的运用型态是否随时间改变，也就是说你的软件在不同的执行阶段有不同的分配/归还形态吗？任何时刻所使用的最大动态分配量（高水位）是多少？自行定义 `operator new` 和 `operator delete` 使我们得以轻松收集到这些信息。

观念上，写一个定制型 `operator new` 十分简单。举个例子，下面是个快速发展得出的初阶段 `global operator new`，促进并协助检测 "overruns"（写入点在分配区块尾端之后）或 "underruns"（写入点在分配区块起点之前）。其中还存在不少小错误，稍后我会完善它。

```
static const int signature = 0xDEADBEEF;
typedef unsigned char Byte;

//这段代码还有若干小错误, 详下。
void* operator new(std::size_t size) throw(std::bad_alloc)
{
    using namespace std;
    size_t realSize = size + 2 * sizeof(int);    //增加大小, 使能够
                                                //塞入两个 signatures.

    void* pMem = malloc(realSize);              //调用 malloc 取得内存.
    if (!pMem) throw bad_alloc();

    //将 signature 写入内存的最前段落和最后段落.
    *(static_cast<int*>(pMem)) = signature;
    *(reinterpret_cast<int*>(static_cast<Byte*>(pMem)
        +realSize-sizeof(int))) = signature;

    //返回指针, 指向恰位于第一个 signature 之后的内存位置.
    return static_cast<Byte*>(pMem) + sizeof(int);
}
```

这个 operator new 的缺点主要在于它疏忽了身为这个特殊函数所应该具备的“坚持 C++ 规矩”的态度。举个例子, 条款 51 说所有 operator new 都应该内含一个循环, 反复调用某个 new-handling 函数, 这里却没有。由于条款 51 就是专门为此协议而写, 所以这儿我暂且忽略之。我现在只想专注于一个比较微妙的主题: 齐位 (alignment)。

许多计算机体系结构 (computer architectures) 要求特定的类型必须放在特定的内存地址上。例如它可能会要求指针的地址必须是 4 倍数 (*four-byte aligned*) 或 doubles 的地址必须是 8 倍数 (*eight-byte aligned*)。如果没有奉行这个约束条件, 可能导致运行期硬件异常。有些体系结构比较慈悲, 没有那么霹雳, 而是宣称如果齐位条件获得满足, 便提供较佳效率。例如 Intel x86 体系结构上的 doubles 可被对齐于任何 byte 边界, 但如果它是 8-byte 齐位, 其访问速度会快许多。

在我们目前这个主题中, 齐位 (alignment) 意义重大, 因为 C++ 要求所有 operator new 返回的指针都有适当的对齐 (取决于数据类型)。malloc 就是在这样的要求下工作, 所以令 operator new 返回一个得自 malloc 的指针是安全的。然而上述 operator new 中我并未返回一个得自 malloc 的指针, 而是返回一个得自 malloc 且偏移一个 int 大小的指针。没人能够保证它的安全! 如果客户端调用 operator new 企图获取足够给一个 double 所用的内存 (或如果我们写个 operator new[], 元素类型是 doubles), 而我们在一部“ints 为 4 bytes 且 doubles 必须 8-byte

齐位”的机器上跑，我们可能会获得一个未有适当齐位的指针。那可能会造成程序崩溃或执行速度变慢。不论哪种情况都非我们所乐见。

像齐位（alignment）这一类技术细节，正可以在那种“因其他纷扰因素而被程序员不断抛出异常”的内存管理器中区分出专业质量的管理器。写一个总是能够运作的内存管理器并不难，难的是它能够优良地运作。一般而言我建议你在必要时才试着写写看。

很多时候是非必要的！某些编译器已经在它们的内存管理函数中切换至调试状态（enable debugging）和标记状态（logging）。快速浏览一下你的编译器文档，很可能就此消除自行撰写 new 和 delete 的需要。许多平台上已有商业产品可以替代编译器自带的内存管理器。如果需要它们来为你的程序提高机能和改善效能，你唯一需要做的就是重新连接（relink）。当然啦，首先你得花点钱买下它们。

另一个选择是开放源码（open source）领域中的内存管理器。它们对许多平台都可用，你可以下载并试试。Boost 程序库（见条款 55）的 Pool 就是这样一个分配器，它对于最常见的“分配大量小型对象”很有帮助。许多 C++ 书籍，包括本书早期版本，都曾展示高效率的小型对象分配器源码，但它们往往漏掉可移植性和齐位考虑、线程安全性……等等令人生厌的麻烦细节。真正称得上程序库者，必然稳健强固。即使你还是执意写一个自己的 news 和 deletes，看一看开放源码版本也可能对若干容易被漠视的细节（它们用来区分“几乎行得通”和“真正行得通”的制品）取得深刻的理解。齐位就是这样一个细节，TR1（见条款 54）支持各类型特定的对齐条件，很值得注意。

本条款的主题是，了解何时可在“全局性的”或“class 专属的”基础上合理替换缺省的 new 和 delete。挖掘更多细节之前，让我先对答案做一些摘要。

- 为了检测运用错误（如前所述）。
- 为了收集动态分配内存之使用统计信息（如前所述）。

- 为了增加分配和归还的速度。泛用型分配器往往（虽然并不总是）比定制型分配器慢，特别是当定制型分配器专门针对某特定类型之对象而设计时。Class 专属分配器是“区块尺寸固定”之分配器实例，例如 Boost 提供的 Pool 程序库便是。如果你的程序是个单线程程序，但你的编译器所带的内存管理器具备线程安全，你或许可以写个不具线程安全的分配器而大幅改善速度。当然，在获得“operator new 和 operator delete 有加快程序速度的价值”这个结论之前，首先请分析你的程序，确认程序瓶颈的确发生在那些内存函数身上。
- 为了降低缺省内存管理器带来的空间额外开销。泛用型内存管理器往往（虽然并非总是）不只比定制型慢，它们往往还使用更多内存，那是因为它们常常在每一个分配区块身上招引某些额外开销。针对小型对象而开发的分配器（例如 Boost 的 Pool 程序库）本质上消除了这样的额外开销。
- 为了弥补缺省分配器中的非最佳齐位（suboptimal alignment）。一如先前所说，在 x86 体系结构上 doubles 的访问最是快速——如果它们都是 8-byte 齐位。但是编译器自带的 operator new 并不保证对动态分配而得的 doubles 采取 8-byte 齐位。这种情况下，将缺省的 operator new 替换为一个 8-byte 齐位保证版，可导致程序效率大幅提升。
- 为了将相关对象成簇集中。如果你知道特定之某个数据结构往往被一起使用，而你又希望在处理这些数据时将“内存页错误”（page faults）的频率降至最低，那么为此数据结构创建另一个 heap 就有意义，这么一来它们就可以被成簇集中在尽可能少的内存页（pages）上。new 和 delete 的“placement 版本”（见条款 52）有可能完成这样的集簇行为。
- 为了获得非传统的行为。有时候你会希望 operators new 和 delete 做编译器附带版没做的某些事情。例如你可能会希望分配和归还共享内存（shared memory）内的区块，但唯一能够管理该内存的只有 C API 函数，那么写下一个定制版 new 和 delete（很可能是 placement 版本，见条款 52），你便得以为 C API 穿上一件 C++ 外套。你也可以写一个自定的 operator delete，在其中将所有归还内存内容覆盖为 0，藉此增加应用程序的数据安全性。

请记住

- 有许多理由需要写个自定的 new 和 delete，包括改善效能、对 heap 运用错误进行调试、收集 heap 使用信息。

条款 51：编写 new 和 delete 时需固守常规

Adhere to convention when writing new and delete.

条款 50 已解释什么时候你会想要写个自己的 operator new 和 operator delete，但并没有解释当你那么做时必须遵守什么规则。这些规则不难奉行，但其中一些并不直观，所以知道它们究竟是些什么很重要。

让我们从 operator new 开始。实现一致性 operator new 必得返回正确的值，内存不足时必得调用 new-handling 函数（见条款 49），必须有对付零内存需求的准备，还需避免不慎掩盖正常形式的 new ——虽然这比较偏近 class 的接口要求而非实现要求。正常形式的 new 描述于条款 52。

operator new 的返回值十分单纯。如果它有能力供应客户申请的内存，就返回一个指针指向那块内存。如果没有那个能力，就遵循条款 49 描述的规则，并抛出一个 bad_alloc 异常。

然而其实也不是非常单纯，因为 operator new 实际上不只一次尝试分配内存，并在每次失败后调用 new-handling 函数。这里假设 new-handling 函数也许能够做某些动作将某些内存释放出来。只有当指向 new-handling 函数的指针是 null，operator new 才会抛出异常。

奇怪的是 C++ 规定，即使客户要求 0 bytes，operator new 也得返回一个合法指针。这种看似诡异的行为其实是为了简化语言其他部分。下面是个 non-member operator new 伪码（pseudocode）：

```
void* operator new(std::size_t size) throw(std::bad_alloc)
{
    //你的 operator new 可能接受额外参数.
    using namespace std;
    if (size == 0) {
        //处理 0-byte 申请.
        size = 1;
        //将它视为 1-byte 申请.
    }
    while (true) {
        尝试分配 size bytes;
```